



standard de mise en forme de requête?

EXERCICES

1. Créer une fonction pl/sql. par deux méthodes différents, calculant le factoriel d'un entier n passé en paramètre.
2. Supposons que la requête suivante retourne plusieurs enregistrements .
`SELECT first_name, ename, sal FROM EMP;`
Implémentez, en bloc PL/SQL, le traitement nécessaire pour afficher la totalité du résultat retournée par la requête donnée précédemment.
3. Créer un déclencheur oracle rassurant que la taille de la table EMP est toujours inférieure ou égale à cent enregistrements.

1)

```
CREATE OR REPLACE FUNCTION factorial (n IN NUMBER)
RETURN NUMBER
IS
BEGIN
    IF n = 0 THEN
        RETURN 1;
    ELSE
        RETURN n * factorial(n-1);
    END IF;
END;
```

```
CREATE OR REPLACE FUNCTION factorial (n IN NUMBER)
RETURN NUMBER
IS
    result NUMBER := 1;
BEGIN
    FOR i IN 1..n
    LOOP
        result := result * i;
    END LOOP;
    RETURN result;
END;
```



```
DECLARE
    result NUMBER;
BEGIN
    result := factorial(5);
    DBMS_OUTPUT.PUT_LINE(result);
END;
```

Statement processed.
6

2)

EMPNO	FIRST_NAME	ENAME	SAL
1	John	Doe	50000
2	Jane	Smith	55000
3	Bob	Johnson	60000

```
DECLARE
    cursor emp_cursor is
        SELECT first_name, ename, sal FROM EMP;
    emp_record emp_cursor%ROWTYPE;
    total_sal NUMBER := 0;
BEGIN
    OPEN emp_cursor;
    LOOP
        FETCH emp_cursor INTO emp_record;
        EXIT WHEN emp_cursor%NOTFOUND;
        DBMS_OUTPUT.PUT_LINE(emp_record.first_name || ' ' || emp_record.ename || ' '
|| emp_record.sal);
        total_sal := total_sal + emp_record.sal;
    END LOOP;
    CLOSE emp_cursor;
    DBMS_OUTPUT.PUT_LINE('Total Salary: ' || total_sal);
END;
```



```
Statement processed.  
John Doe 50000  
Jane Smith 55000  
Bob Johnson 60000  
Total Salary: 165000
```

3)

```
CREATE OR REPLACE TRIGGER check_emp_size  
BEFORE INSERT OR UPDATE OR DELETE ON EMP  
FOR EACH ROW  
DECLARE  
    emp_count INTEGER;  
BEGIN  
    SELECT COUNT(*) INTO emp_count FROM EMP;  
    IF emp_count >= 100 THEN  
        RAISE_APPLICATION_ERROR(-20001, 'la table EMP est pleine, impossible  
d''insérer plus d''enregistrements');  
    END IF;  
END;
```





EXERCICE

En supposant l'existence de la table EMPLOYE dont la structure est définie comme suit :

EMPLOYE(empno,ename,job,manager,salaire,loc).

I. Réalisez les requêtes permettant d'afficher :

- 1) Le numéro et le nom de l'employé ayant le plus grand salaire. *select numero, nom from Employee where salaire >= (select max(salaire) from Employee)*
- 2) Les numéros et noms des employés dont chacun a aussi la même localité que son manager. *select numero, nom from Employee where loc like manager*
- 3) Combien des fois se répète, dans la table EMPLOYE, chaque localité. *select loc, count(*) from Employee group by loc*
- 4) Ecrire un bloc PL/SQL permettant d'afficher le contenu de la table EMPLOYE en utilisant les curseurs d'oracle.

II. Créer un déclencheur conçu dans le but d'interdire toute opération de diminution de salaire.

1)

EMPNO	ENAME	JOB	MANAGER	SALAIRE	LOC
1	John Doe	Manager	-	6000	Paris
2	Jane Smith	Developer	1	5000	Paris
3	Bob Johnson	Developer	1	4500	Lyon
4	Alice Brown	Developer	2	4000	Paris

1)

```
SELECT empno, ename FROM EMPLOYE WHERE loc = (SELECT loc FROM EMPLOYE WHERE empno = manager);
```

EMPNO	ENAME
1	John Doe

2)

```
SELECT empno, ename FROM EMPLOYE WHERE loc = (SELECT loc FROM EMPLOYE WHERE empno = manager);
```



3)

```
SELECT loc, COUNT(*) FROM EMPLOYE GROUP BY loc;
```

LOC	COUNT(*)
Paris	3
Lyon	1

4)

```
DECLARE
  CURSOR employee_cursor IS SELECT empno, ename, job, manager, salaire, loc FROM
EMPLOYE;
  employee_record employee_cursor%ROWTYPE;
BEGIN
  OPEN employee_cursor;
  LOOP
    FETCH employee_cursor INTO employee_record;
    EXIT WHEN employee_cursor%NOTFOUND;
    DBMS_OUTPUT.PUT_LINE(employee_record.empno || ' ' || employee_record.ename ||
' ' || employee_record.job || ' ' || employee_record.manager || ' ' ||
employee_record.salaire || ' ' || employee_record.loc);
  END LOOP;
  CLOSE employee_cursor;
END;
```

```
Statement processed.
1 John Doe Manager 6000 Paris
2 Jane Smith Developer 1 5000 Paris
3 Bob Johnson Developer 1 4500 Lyon
4 Alice Brown Developer 2 4000 Paris
```



||)

```
CREATE OR REPLACE TRIGGER prevent_decrease
BEFORE UPDATE OF salaire ON EMPLOYE
FOR EACH ROW
BEGIN
    IF :new.salaire < :old.salaire THEN
        RAISE_APPLICATION_ERROR(-20001, 'Il n''est pas autorisé de diminuer le
salaire');
    END IF;
END;
```

```
UPDATE EMPLOYE
SET salaire = 5000
WHERE empno = 1;
```

```
ORA-20001: Il n'est pas autorisé de diminuer le salaire ORA-06512: at "SQL_YCPCFMGEKANI"
ORA-06512: at "SYS.DBMS_SQL", line 1721
```



- Convertir une date en chaîne de caractères.
- Convertir une chaîne de caractères en date.

Exercice 1 (6 points).

1. Ecrire une Fonction en PL\SQL nommée factory qui calcule le factoriel d'un nombre donnée par l'utilisateur (2 point).
2. Ecrire un Programme PL\SQL permettant de stocker dans trois variables le résultat d'exécution de la requête suivante : `SELECT nom, prenom, date_naissance FROM Personnes`, en prenant en compte les contraintes suivantes (4 point) :
 - Les types des variables doivent être identiques à celles des colonnes.
 - Les exceptions doivent être gérées quand elles sont soulevées dans votre code,
 - Les valeurs des variables doivent être affichées à la fin du programme.

1)

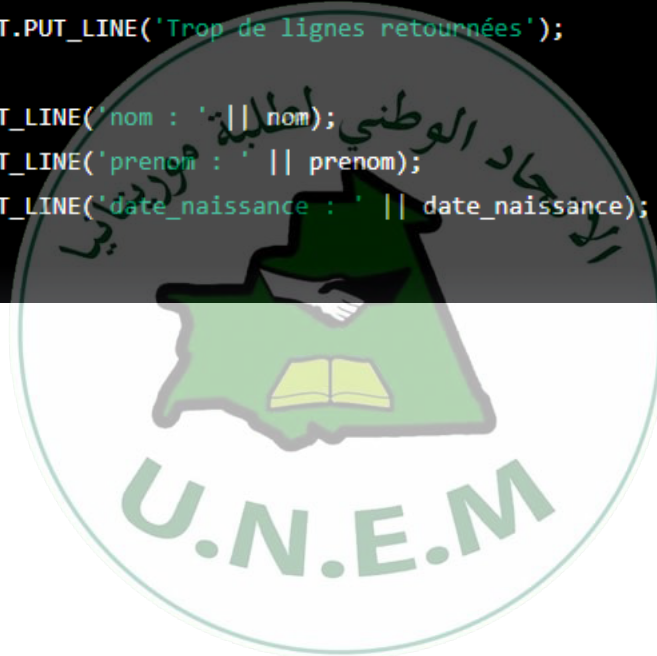
```
CREATE OR REPLACE FUNCTION factory(n NUMBER)
RETURN NUMBER
AS
    factorial NUMBER := 1;
BEGIN
    FOR i IN 1..n LOOP
        factorial := factorial * i;
    END LOOP;
    RETURN factorial;
END;
```

```
DECLARE
    result NUMBER;
BEGIN
    result := factory(5);
    DBMS_OUTPUT.PUT_LINE('Result : ' || result);
END;
```

```
Statement processed.
Result : 120
```




```
DECLARE
    nom Personnes.nom%TYPE;
    prenom Personnes.prenom%TYPE;
    date_naissance Personnes.date_naissance%TYPE;
BEGIN
    SELECT nom, prenom, date_naissance INTO nom, prenom, date_naissance
    FROM Personnes;
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('Aucune donnée trouvée');
    WHEN TOO_MANY_ROWS THEN
        DBMS_OUTPUT.PUT_LINE('Trop de lignes retournées');
END;
DBMS_OUTPUT.PUT_LINE('nom : ' || nom);
DBMS_OUTPUT.PUT_LINE('prenom : ' || prenom);
DBMS_OUTPUT.PUT_LINE('date_naissance : ' || date_naissance);
END;
```





3. Que signifie le terme schéma en oracle ?
4. Quelle particularité a la base des données par défaut ?

EXERCICE I

Supposons que la table, présenté ci-dessous, est déjà créée dans notre base des données, Employes (matricule, ename, firstname, salaire, daterecrutement) **Remarque :** dans tout l'énoncé de l'exercice, les requêtes de sélection utilisées sont de type **SELECT INTO**,

1. Créer une procédure stockée qui affiche les informations de l'employé qu'on y fournit
2. Créer une fonction qui calcule la masse salariale de tous les employés.
3. Ecrire un bloc PL/SQL pour afficher éventuellement le matricule et le salaire de tous les employés sans utiliser la technique des curseurs.
4. En utilisant l'option de curseurs, écrire un programme PL/SQL visant à afficher l'intégralité des données contenues dans la table Employes.

1)

MATRICULE	ENAME	FIRSTNAME	SALAIRE	DATERECRUTEMENT
1	John	Doe	50000	01-JAN-10
2	Jane	Smith	55000	01-JAN-11
3	Bob	Johnson	60000	01-JAN-12
4	Alice	Brown	65000	01-JAN-13

```
CREATE OR REPLACE PROCEDURE affiche_employe (p_matricule NUMBER)
AS
    nom Employees.ename%TYPE;
    prenom Employees.firstname%TYPE;
    salaire Employees.salaire%TYPE;
    date_recrutement Employees.daterecrutement%TYPE;
BEGIN
    SELECT ename, firstname, salaire, daterecrutement INTO nom, prenom, salaire,
    date_recrutement
    FROM Employees
    WHERE matricule = p_matricule;
    DBMS_OUTPUT.PUT_LINE('Informations de l''employé :');
    DBMS_OUTPUT.PUT_LINE('Matricule : ' || p_matricule);
    DBMS_OUTPUT.PUT_LINE('Nom : ' || nom);
    DBMS_OUTPUT.PUT_LINE('Prénom : ' || prenom);
    DBMS_OUTPUT.PUT_LINE('Salaire : ' || salaire);
    DBMS_OUTPUT.PUT_LINE('Date de recrutement : ' || date_recrutement);
END;
```



```
affiche_employe(1);  
END;
```

```
Statement processed.  
Informations de l'employé :  
Matricule : 1  
Nom : John  
Prénom : Doe  
Salaire : 50000  
Date de recrutement : 01-JAN-10
```

2)

```
CREATE OR REPLACE FUNCTION masse_salariale  
RETURN NUMBER  
AS  
    total_salaire NUMBER := 0;  
BEGIN  
    SELECT SUM(salaire) INTO total_salaire FROM Employes;  
    RETURN total_salaire;  
END;
```

```
DECLARE  
    total_salaire NUMBER;  
BEGIN  
    total_salaire := masse_salariale();  
    DBMS_OUTPUT.PUT_LINE('Total salaire: ' || total_salaire);  
END;  
/
```

```
Statement processed.  
Total salaire: 230000
```



```
DECLARE
    matricule Employes.matricule%TYPE;
    salaire Employes.salaire%TYPE;
BEGIN
    FOR employe IN (SELECT matricule, salaire FROM Employes)
    LOOP
        DBMS_OUTPUT.PUT_LINE('Matricule : ' || employe.matricule);
        DBMS_OUTPUT.PUT_LINE('Salaire : ' || employe.salaire);
    END LOOP;
END;
```

```
Statement processed.
Matricule : 1
Salaire : 50000
Matricule : 2
Salaire : 55000
Matricule : 3
Salaire : 60000
Matricule : 4
Salaire : 65000
```

4)

```
DECLARE
    CURSOR c_employees IS SELECT matricule, ename, firstname, salaire,
    daterecrutement FROM Employes;
    v_matricule Employes.matricule%TYPE;
    v_ename Employes.ename%TYPE;
    v_firstname Employes.firstname%TYPE;
    v_salaire Employes.salaire%TYPE;
    v_daterecrutement Employes.daterecrutement%TYPE;
BEGIN
    OPEN c_employees;
    LOOP
        FETCH c_employees INTO v_matricule, v_ename, v_firstname, v_salaire,
        v_daterecrutement;
        EXIT WHEN c_employees%NOTFOUND;
        DBMS_OUTPUT.PUT_LINE(v_matricule || ' - ' || v_ename || ' - ' || v_firstname
        || ' - ' || v_salaire || ' - ' || v_daterecrutement);
    END LOOP;
    CLOSE c_employees;
END;
```

```
Statement processed.
1 - John - Doe - 50000 - 01-JAN-10
2 - Jane - Smith - 55000 - 01-JAN-11
3 - Bob - Johnson - 60000 - 01-JAN-12
4 - Alice - Brown - 65000 - 01-JAN-13
```



l'intégralité des données contenues dans la table emploie, ...

EXERCICE II

1. Vous connecter en tant qu'utilisateur disposant du rôle « sysdba »
2. Créer un compte « stage » disposant d'un mot de passe aussi « stage ».
3. Accorder au compte créé précédemment les droits et les rôles suivants :
Connect, create session et resource.
4. Retirer du même compte créé déjà les mêmes droits et rôles qui lui sont déjà affectés récemment.

BONNES CHANCES

الاتحاد الوطني لطلبة
U.N.E.M موريتانيا

```
-- 1. Connect as sysdba  
CONNECT / AS SYSDBA
```

```
-- 2. Create a new account 'stage' with password 'stage'  
CREATE USER stage IDENTIFIED BY stage;
```

```
-- 3. Grant 'stage' account the following privileges: connect, create session,  
and resource  
GRANT connect, create session, resource TO stage;
```

```
-- 4. Revoke the same privileges from 'stage' account
```

```
REVOKE create session FROM stage;
```

```
REVOKE resource FROM stage;
```



Exercice I

Soit le schéma relationnel suivant :

Clients (no_cli, nom, prenom, tel, adresse) **Comptes_retrait_carte** (no_cpt, devise, solde, cli) **Transaction_autorisee** (ref, no_cpt, start_time, mnt_retire)

Conditions : la table Transaction_autorisee contient les informations relatives aux retraits réalisés par carte bancaire, tout en tenant compte que la carte bancaire ne peut pas retirer plus de cent mille ouguiyas dans les dernières vingt-quatre heures, **start time** : date à partir de laquelle la carte est autorisée à retirer le plafond (cent mille ouguiya),

mnt retire : cumul des montants retirés par une carte avant l'écoulement de vingt-quatre heures.

1. Décrire le processus d'établissement de connexion à une base des données oracle en présentant, dans le bon ordre, la série des commandes à exécuter.
2. En appliquant un traitement procédural, faites le nécessaire pour récupérer la liste des clients dont les comptes de devise sont en euro.
3. Quel traitement faut-il appliquer pour faire respecter les règles liées aux retraits par carte bancaire.

1)

1. Pour établir une connexion à une base de données Oracle, les commandes à exécuter dans l'ordre sont :

- Ouvrir le client SQLplus
- Se connecter en utilisant la commande : `CONNECT nom_utilisateur/mot_de_passe@nom_instance`

2)

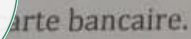
```
SELECT nom, prenom
FROM Clients
WHERE no_cli IN (SELECT cli FROM Comptes_retrait_carte WHERE devise = 'EUR');
```



```
CREATE OR REPLACE TRIGGER check_withdrawal_limit
BEFORE INSERT ON Transaction_autorisee
FOR EACH ROW
DECLARE
    total_withdrawn NUMBER;
BEGIN
    -- Retrieve the total amount withdrawn by the card in the last 24 hours
    SELECT SUM(mnt_retire) INTO total_withdrawn
    FROM Transaction_autorisee
    WHERE no_cpt = :NEW.no_cpt
    AND start_time >= SYSDATE - INTERVAL '1' DAY;

    -- Compare the total withdrawn to the limit
    IF total_withdrawn + :NEW.mnt_retire > 100000 THEN
        RAISE_APPLICATION_ERROR(-20001, 'Withdrawal limit exceeded');
    END IF;
END;
```





Remarque : utilisez la fonction `put()` à la place de `put_line()`

```
DECLARE
  n NUMBER := &n; -- saisir le niveau souhaité
BEGIN
  FOR i IN 0..n-1 LOOP
    FOR j IN 0..i LOOP
      DBMS_OUTPUT.PUT('0');
    END LOOP;
    DBMS_OUTPUT.PUT_LINE(' Niveau '||i);
  END LOOP;
END;
```

```
Statement processed.  
0 Niveau 0  
00 Niveau 1  
000 Niveau 2  
0000 Niveau 3  
00000 Niveau 4  
000000 Niveau 5  
0000000 Niveau 6  
00000000 Niveau 7  
000000000 Niveau 8  
0000000000 Niveau 9
```




QUESTION DU COUR

- Pourquoi , pour des raisons de sécurité ,

vaut-t-il mieux d'utiliser un nom de service que d'utiliser un nom de base des données ?

- utiliser un nom de service plutôt qu'un nom de base de données peut contribuer à renforcer

la sécurité de votre base de données en rendant plus difficile l'accès non autorisé et en masquant les détails de votre architecture de base de données.

- Quelle particularité a la base des données par défaut ?

- C'est une base de données relationnelle: elle utilise un schéma relationnel

pour stocker les données,

C'est une base de données performante: elle utilise une technologie de gestion de mémoire et de stockage avancée

- Expliquez , en bref , le manque de souplesse susceptible d'être envisagé lors d'exécution des

requêtes de type SELECT INTO FROM:

- Le manque de souplesse lors de l'exécution de requêtes de type SELECT INTO peut être dû à plusieurs raisons.

Tout d'abord, cette requête ne permet généralement pas de sélectionner des colonnes spécifiques dans les résultats,

mais plutôt toutes les colonnes de la table ou de la vue spécifiée.

- Pour remédier au même problème présenté à la question N ° 1 . est-t-il nécessaire d'implémenter

le mécanisme de gestion d'exceptions , si oui comment sera fait ?

- Il n'est pas nécessaire d'implémenter explicitement le mécanisme de gestion d'exceptions pour



lié au manque de souplesse lors de l'exécution de requêtes de type SELECT
O. Cependant,

si vous souhaitez gérer des erreurs qui peuvent survenir lors de l'exécution de cette requête,

vous pouvez utiliser un bloc de gestion d'exceptions.

Voici comment cela pourrait être fait :

BEGIN

SELECT col1, col2, col3 INTO new_table

FROM existing_table;

EXCEPTION

WHEN OTHERS THEN

-- traiter l'erreur ici

END;

- Pour quoi les pluparts SGBDR intègrent dans leurs offres la PL/SQL ?
- ✓ PL/SQL est un langage de **programmation** très puissant et utile pour les développeurs de bases de données

qui souhaitent créer des applications de base de données efficaces et fiables. C'est pour cette raison

que la plupart des SGBDR intègrent PL/SQL dans leurs offres.

- Comment peut - on définir , au niveau du système oracle , un mode d'authentification géré

par le système d'exploitation ?

- ✓ Pour définir un mode d'authentification géré par le système d'exploitation dans Oracle,

vous devez utiliser l'authentification externe.



- Citer trois structures de stockage logique en oracle:
- ✓ Tablespaces
- ✓ Segments
- ✓ Extents

- Quelle est la difference entre un tablespace permanent et un tablespace temporaire:

- ✓ Tablespaces permanents : Un tablespace permanent est un tablespace

qui stocke les objets de base de données de manière permanente.

- ✓ Tablespaces temporaires : Un tablespace temporaire est un tablespace

qui stocke les données temporaires générées par les requêtes SQL.

- Donner un cas d'utilisation d'un tablespace temporaire:

- ✓ `SELECT * FROM employees`

`WHERE salary > 50000`

`ORDER BY salary ASC`

`INTO TEMPORARY TABLE temp_employees;`

- AutoCommit : détermine si les transactions sont automatiquement validées lorsqu'elles sont terminées,

- ✓ ou si elles doivent être explicitement validées par l'utilisateur.

- Linesizes : détermine la taille des lignes affichées dans le résultat d'une requête SQL.

- Pagesizes : détermine la taille des pages utilisées pour afficher les résultats d'une requête.



➤ Écrire la syntaxe de la commande Oracle pour modifier la valeur d'un paramètre:

- ✓ ALTER SYSTEM SET parameter_name = parameter_value;
- ✓ ALTER SYSTEM SET Linesizes = 200;

- Citer les fonctions de conversion explicites dont les rôles sont indiqués comme suit :

- ✓ Convertir un nombre en chaîne de caractères : TO_CHAR
- ✓ Convertir une chaîne de caractère en nombre : TO_NUMBER
- ✓ Convertir une date en chaîne de caractères : TO_CHAR
- ✓ Convertir une chaîne de caractères en date : TO_DATE

- Type de structure de données :

- ✓ Il s'agit des différents formats dans lesquels les données peuvent

être stockées et organisées, tels que les tableaux, les listes...

- -Un programme client envoie des requêtes à un programme serveur, qui traite ces requêtes et renvoie les résultats.

- Objet mémoire : Un objet mémoire est une zone de mémoire peuvent être utilisés pour stocker des variables, des tableaux...

- Session : Une session est une période de temps pendant laquelle un utilisateur interagit avec un système

- Un curseur Oracle est utilisé pour parcourir et manipuler les enregistrements d'une table ou d'une vue dans une base de données Oracle.